



Backend Engineer Assignment — Ira AI Gateway



Duration: 72–96 hours

Audience: Backend Engineer with 1–5 years experience

Goal: Evaluate end-to-end thinking: backend system design, distributed design, async implementation, safety handling, and DevOps execution.

Context

Ira is an AI companion chatbot used by millions across WhatsApp and our own App.

The backend must support different subscription tiers (**free, premium, enterprise**), deliver reliable performance for higher tiers, and degrade gracefully for lower tiers—without sounding technical or robotic.

The goal of this assignment is to design and implement the architecture that powers Ira's message processing pipeline: data stores, rate limiting, routing, worker pools, safety handling, and analytics.

Technical Constraints

- **Python 3.13** (required)
- **FastAPI** with uvicorn
- **MongoDB 7+** with 4 collections (1M documents each)
- **Redis** for distributed state
- **Server-based deployment** (Docker Compose simulating multi-node)

- **Custom tier-aware load balancer**
 - All I/O must be **non-blocking** (asyncio)
-

Part 1 — Data Modelling & Query Design

You will design the database layer, including document schemas, indexing strategy, and performance-critical queries.

Collections

There are **four main collections**:

1. **users**
2. **personalities** (Ira's per-user personality model)
3. **sessions** (you define what a "session" means in your system)
4. **messages**

Requirements

- ☐ Design and document your schemas
- ☐ Create seed data (~1 million documents per collection; realistic distribution of users & activity)
- ☐ Design **compound indexes** to support your primary queries
- ☐ Implement query patterns using **\$lookup** such as:
 - Fetching the current active session for a user
 - Retrieving recent messages for contextual LLM calls
 - Aggregations by tier or activity
- ☐ Include performance notes (e.g., explain why your indexes help)
- ☐ Provide `explain("executionStats")` output for critical queries



Performance Target: Queries 1 & 2 (hot path) must achieve **< 50ms** on 1M documents

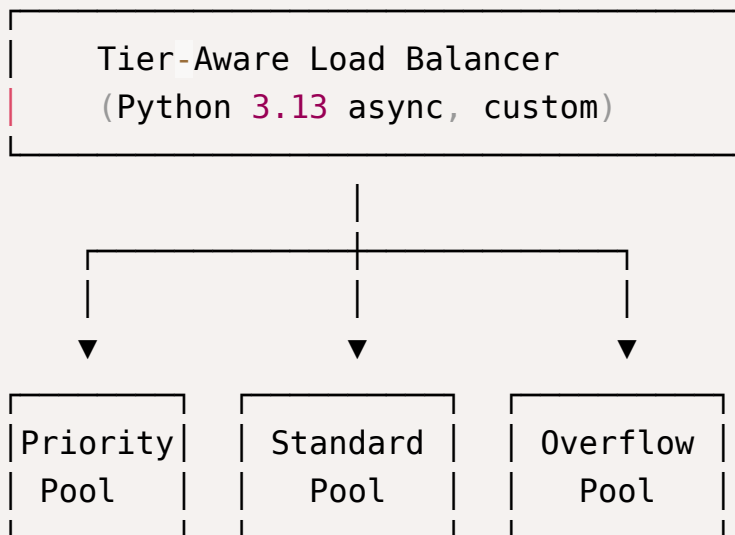
Part 2 — Tier-Aware Load Balancer

You will design and implement a **custom load balancer** that routes incoming chat requests to different worker pools depending on:

- User tier
- Pool health
- Current system load
- Latency constraints
- Fairness and graceful degradation

You are free to choose how the routing logic works (queueing strategies, concurrency primitives, health scoring, etc.).

Architecture Concept



Requirements

- ☐ Define worker pools (priority, general, overflow — or any design you choose)
 - ☐ Implement:
 - Pool health tracking
 - Routing based on tier
 - Load shedding strategies
 - ☐ **Enterprise** traffic should remain stable even under extreme load
 - ☐ **Premium** traffic should degrade more slowly than free traffic
 - ☐ **Free** traffic should degrade gracefully, not abruptly
-

Part 3 — Graceful Rate Limiting & Safety Layer



Critical Requirement: Ira should **never** return harsh system messages. Rate limiting and safety filtering should feel natural and personality-aware.

Rate Limiting Requirements

Implement a **graceful rate-limiting** mechanism that:

- ☐ Detects users approaching or exceeding limits (per your design)
- ☐ Sends **personality-aware responses** for the first limiting event
- ☐ Avoids spamming the user with repeated system messages
- ☐ Handles subsequent limits with defined behavior (silence, queueing, or your design)

Safety Layer Requirements

Build a **blocking system** that:

- ☐ Detects jailbreak attempts, NSFW content, or unsafe prompts
- ☐ Returns **personality-aware** responses, not mechanical errors
- ☐ Integrates with the rate limiting flow

You can use any data structures or persistence strategy you judge appropriate.

Part 4 — Non-Blocking Analytics & Structured Logging

You will introduce **observability** into the system without slowing down user-facing flows.

Analytics Pipeline Requirements

- ☐ Accepts events **without blocking** critical paths
- ☐ Uses a **bounded queue** and drops intelligently under pressure
- ☐ Flushes batches in the background
- ☐ Track calls must return in **< 1ms**

Structured Logging Requirements

- ☐ **Correlation IDs** for request tracing
- ☐ Context in every log (tier, user ID, operation type, etc.)
- ☐ **Slow-operation detection** (auto-log operations > threshold)
- ☐ Non-blocking writes

Graceful Shutdown

- ☐ The system must shut down gracefully, flushing pending analytics/logs
 - ☐ No data loss during normal shutdown
-

Part 5 — End-to-End Integration, Testing & DevOps

You will assemble the full system into a **runnable environment**.

Docker Compose Setup

Build a complete environment including:

- ☐ Your application services (load balancer, worker pools)
- ☐ MongoDB with 4M seeded documents
- ☐ Redis for distributed state
- ☐ Any background worker services
- ☐ Seeder script

Testing Requirements

Write **pytest-based** load and functional tests to simulate:

- ☐ Normal load scenario
- ☐ High load scenario
- ☐ Rate limiting behavior (verify graceful messages)
- ☐ Safety blocking flows
- ☐ Worker pool overload scenarios

Test Report

Provide:

- ☐ Latency measurements (p50, p99 per tier)
 - ☐ Behavior observations
 - ☐ Notes on trade-offs
-

Final Deliverables

Your submission should include:

1. High-Level System Design

Architectural diagram (Figma, Excalidraw, Miro, etc.) showing:

- Data flow
- Worker pools
- Load balancer
- Rate limiting layer
- Safety layer
- Observability components

2. Low-Level Design

- Components, classes, interfaces
- Queueing/concurrency model
- State diagrams for rate limiting & safety blocking
- Request lifecycle explanation

3. Working Implementation

- ☐ Fully runnable via `docker-compose up`
- ☐ Async Python 3.13 (FastAPI or similar)
- ☐ All major components present:
 - Load balancer
 - Worker pools
 - Rate limiting
 - Safety filters
 - Analytics/logging

- Database integration

☐ Test suite

4. Tests

☐ Pytest load tests

☐ Unit tests for core logic

☐ Behavioral tests for tiered routing, rate limiting, and safety rules

5. README

Include:

- Setup instructions
 - Design decisions
 - Trade-offs
 - How to run tests
 - Anything else that helps reviewers understand your system
-

What to Submit

1. **Private GitHub repository** link
 2. Add collaborators: `vatsalbharti`, `auchi-rumik`, `sahil@rumik.ai`
 3. **Load test results** (screenshots or exports)
 4. **Design document** with trade-offs explained
 5. **Video walkthrough** (optional, 5-10 min)
-



Note for Candidates

This is not about building a production-level platform.

We want to see **how you think**, design, and implement systems under real constraints.

Clarity of reasoning, correctness of architecture, and code quality matter most.